

UNIVERSITÉ DE SHERBROOKE
Faculté de génie
Département de génie électrique et génie informatique

RAPPORT : APP2

APP2

Présenté à
Philippe Gournay

Présenté par
Équipe numéro 8
Felix Boivin – BOIF1302
Mathieu Desautels – DESM1210

Sherbrooke – 29 septembre 2026

TABLE DES MATIÈRES

1.	Introduction	2
2.	Explication du formatage de l'image	2
3.	Technique vectorielle	3
3.1	Schéma-bloc	3
3.2	Explication du codeur	3
3.3	Explication du décodeur	4
3.4	Allocation du budget de bits	4
4.	Technique différentielle	5
4.1	Schéma-bloc	5
4.2	Explication du codeur	5
4.3	Explication du décodeur	6
4.4	Allocation du budget de bits	7
5.	Technique BTC	7
5.1	Schéma-bloc	7
5.2	Explication du codeur	7
5.3	Explication du décodeur	8
5.4	Allocation du budget de bits	8
6.	Conclusion	9
7.	Images de comparaison Lenna	10

LISTE DES FIGURES

Figure 1 : Schéma bloc technique vectoriel	3
Figure 2 : Schéma bloc technique différentiel	5
Figure 3 : Schéma bloc technique BTC	7
Figure 4 : Lenna version DPCM	10
Figure 5 : Lenna version vectorielle	10
Figure 6 : Lenna version BTC	10

LISTE DES TABLEAUX

Tableau 1 : Tableau comparatif finale	9
---------------------------------------	---

1. INTRODUCTION

Dans le cadre d'un contrat avec la compagnie HyperMédia, un codec d'images doit être conçu dans le but de réduire l'utilisation des ressources de calcul, de stockage et de transmission. Les spécifications demandées par le superviseur sont les suivantes :

1. Prendre en entrée des images matricielles RVB de profondeur 24 bits ou en niveaux de gris de profondeur 8 bits, et donner en sortie des images en niveaux de gris de profondeur 8 bits
2. Prendre en entrée des images matricielles de dimensions variables et donner en sortie des images de 256 x 256 pixels
3. Réduire la profondeur de couleur et redimensionner l'image à 256 x 256 pixels lorsque nécessaire, avant d'appliquer la quantification
4. Maximiser le PSNR
5. Respecter un débit maximal de 5 bits/pixel pour l'image encodée, métadonnées incluses.

2. EXPLICATION DU FORMATAGE DE L'IMAGE

Chaque image doit être standardisée avec des dimensions de 256 x 256 pixels et doit être représentée en 256 niveaux de gris. Cependant, les images d'entrée peuvent varier en dimension et certaines peuvent être en couleur.

Premièrement, si l'image est en couleur, elle doit être convertie de RVB en niveaux de gris. Pour ce faire, la matrice de l'image source est analysée et si elle compte trois couches (rouge, vert et bleu) l'opération suivante est réalisée.

$$NiveauxGris = \frac{R + V + B}{3}$$

Ensuite, il est possible que les images sources soient plus grandes ou plus petites que le format voulu de 256 x 256 pixels pour le traitement. La technique du formatage par interpolation bilinéaire est idéale pour ce mandat. Cette méthode détermine la valeur d'un pixel en se comparant aux pixels voisins et en faisant ensuite une double interpolation linéaire, soit horizontalement et verticalement. Cette méthode propose un avantage par rapport à la méthode du plus proche voisin (PPV), car l'interpolation produit un lissage qui permet de mitiger les effets de crénelage.

3. TECHNIQUE VECTORIELLE

3.1 SCHÉMA-BLOC

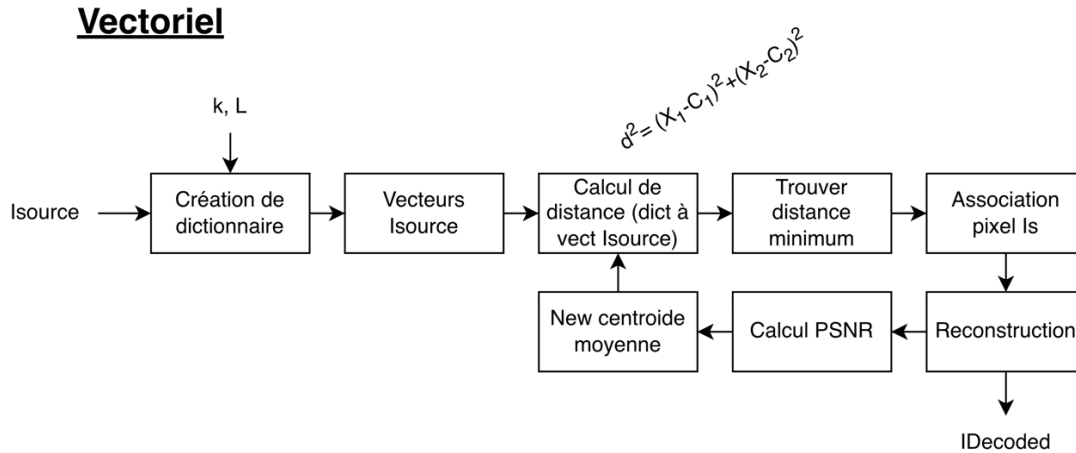


Figure 1 : Schéma bloc technique vectoriel

3.2 EXPLICATION DU CODEUR

Le codeur vectoriel commence par subdiviser l'image source en blocs de taille au choix. Dans ce cas, les blocs sont de 2 x 1 pixels, ce qui donne 32 768 vecteurs de dimension 2. Ensuite, un dictionnaire de $K = 2^n$ vecteurs est conçu à l'aide de l'algorithme de Linde-Buzo-Gray (LBG) avec la technique de division (*splitting*). Avec $n = 9$ bits, le dictionnaire contient 512 vecteurs et forme une matrice de 2 x 512.

Le dictionnaire initial contient un seul vecteur, soit la moyenne de tous les blocs de l'image. Chaque vecteur y du dictionnaire est ensuite dédoublé en y et $y + \varepsilon$, où ε est un vecteur de perturbation aléatoire. Le nouveau dictionnaire est raffiné par itérations successives. Premièrement, chaque bloc x de l'image est associé au vecteur du dictionnaire le plus proche selon la distance euclidienne au carré :

$$d^2 = (x_1 - c_1)^2 + (x_2 - c_2)^2$$

Ensuite, chaque vecteur du dictionnaire est remplacé par le centroïde des blocs qui lui sont associés, c'est-à-dire leur moyenne. La distorsion moyenne D est calculée à chaque itération.

$$D = \left(\frac{1}{N}\right) \sum \|x_n - y_i\|^2$$

Les itérations cessent lorsque la variation relative de la distorsion devient inférieure à un seuil, soit $\frac{D^{(k-1)} - D^{(k)}}{D^{(k)}} < 10^{-3}$. Le dédoublement est répété jusqu'à l'obtention des 512 vecteurs, et les valeurs du dictionnaire final sont arrondies à l'entier le plus proche afin d'être représentées sur 8 bits.

Finalement, chaque bloc de l'image est associé au vecteur du dictionnaire final ayant la plus petite distance. Seul l'indice de ce vecteur, représenté sur 9 bits, est conservé. Les indices constituent les données et le dictionnaire constitue les métadonnées.

3.3 EXPLICATION DU DÉCODEUR

Le décodeur reçoit les indices de chaque bloc ainsi que le dictionnaire. Pour chaque bloc, l'indice i reçu permet de retrouver directement le vecteur correspondant dans le dictionnaire :

$$\hat{x} = y_i$$

Il s'agit d'une simple consultation de table, ce qui rend le décodeur très rapide. Ensuite, chaque vecteur de deux valeurs est remis sous la forme d'un bloc de 2 x 1 pixels et replacé à sa position d'origine. Pour une image de 256 x 256 pixels, les blocs forment une grille de 128 x 256 blocs. On retrouve ensuite l'image synthétisée de grandeur originale en juxtaposant chacun des blocs reconstruits.

3.4 ALLOCATION DU BUDGET DE BITS

L'image d'entrée est de taille 256 x 256 pixels ce qui donne un total de 65 536 pixels. Pour la méthode vectoriel les bits sont transmis sous la forme de 2x1 pixel, cela donne donc un nombre total de 32 768 blocs. Chaque vecteur de 2x1 est remplacé par un index dans un dictionnaire 512 codes tous avec des valeurs de 9 bits. Il y a donc 294 912 bits de données par images ($32\,768 \cdot 9 = 294\,912$). Ce qui en est des métadonnées le décodeur a besoin de recevoir le dictionnaire composé de 512 emplacements contenant 2 valeurs chacune représentée sur 8 bits pour un total de 8 192 bits par image ($512 \cdot 2 \cdot 8 = 8192$). Le total du nombre de bits est donc de 303 104 bits ($294\,912 + 8192 = 303\,104$).

$$R = \frac{\text{Taille totale en bits}}{\text{Nombre total de pixels}} = \frac{303\,104}{65\,536} = 4,625 \text{ bits/pixel}$$

4. TECHNIQUE DIFFÉRENTIELLE

4.1 SCHÉMA-BLOC

Différentiel

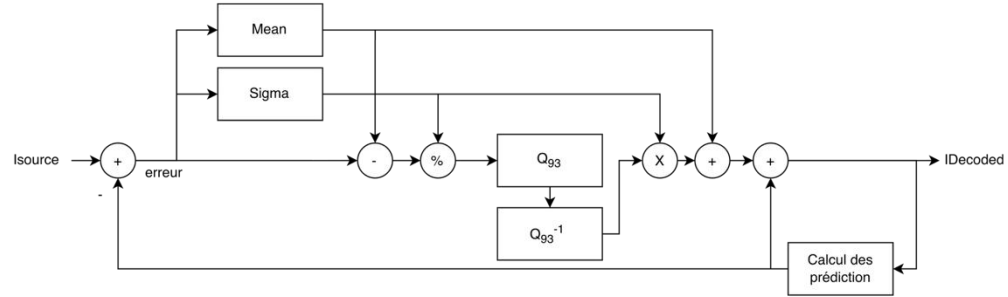


Figure 2 : Schéma bloc technique différentiel

4.2 EXPLICATION DU CODEUR

Le codeur DPCM traite l'image pixel par pixel, de gauche à droite et de haut en bas. Pour chaque pixel, une prédiction $p[j, k]$ est calculée à partir des pixels voisins déjà reconstruits $\hat{x}$. Pour les pixels intérieurs, le prédicteur médian est utilisé :

$$p_1 = \frac{\hat{x}[j - 1, k] + \hat{x}[j, k - 1]}{2}$$

$$p_2 = \frac{\hat{x}[j - 1, k - 1] + \hat{x}[j, k - 1]}{2}$$

$$p_3 = \frac{\hat{x}[j - 1, k - 1] + \hat{x}[j - 1, k]}{2}$$

$$p[j, k] = \text{médiane}\{p_1, p_2, p_3\}$$

Pour les pixels de bordure, un prédicteur simple est utilisé : le pixel reconstruit de gauche pour la première ligne, le pixel reconstruit du dessus pour la première colonne, et la valeur 128 pour le premier pixel. Ensuite, l'erreur de prédiction est calculée :

$$e[j, k] = x[j, k] - p[j, k]$$

Cette erreur est normalisée à l'aide de sa moyenne μ_e et de son écart-type σ_e , estimés à partir d'une première prédiction effectuée sur l'image source :

$$e_n = \frac{(e - \mu_e)}{\sigma_e}$$

La normalisation permet d'utiliser directement le pas de quantification normalisé optimal du tableau 9.3 du livre de référence, calculé pour une variance unitaire. Puisque l'erreur de prédiction est concentrée autour de zéro, une densité de probabilité laplacienne est supposée. L'erreur normalisée est quantifiée par un quantificateur uniforme à $N = 2^n$ niveaux. Avec $n = 5$ bits, $N = 32$ et $\Delta = 0,2799$. Les seuils de décision sont espacés de Δ et centrés sur zéro :

$$b_i = i \cdot \Delta, i = -N/2 + 1, \dots, N/2 - 1$$

auxquels s'ajoutent $-\infty$ et $+\infty$ aux extrémités, afin que toute valeur soit associée à un intervalle. Chaque erreur reçoit le code de l'intervalle dans lequel elle se trouve.

Finalement, l'encodeur contient un décodeur local : il reconstruit chaque pixel de la même façon que le décodeur, soit $\hat{x} = p + \hat{e}$. Les prédictions suivantes utilisent ainsi exactement la même information que le décodeur, ce qui évite l'accumulation de l'erreur de quantification.

4.3 EXPLICATION DU DÉCODEUR

Le décodeur reçoit la matrice des codes ainsi que la moyenne μ_e et l'écart-type σ_e de l'erreur de prédiction. Le nombre de bits et le pas Δ étant fixés, le décodeur recalcule lui-même les niveaux de reconstruction, situés au centre de chaque intervalle de décision :

$$y_i = (2i - N + 1) \cdot \Delta / 2, i = 0, \dots, N - 1$$

Par exemple, pour un intervalle compris entre les seuils 0 et Δ , le niveau de reconstruction est $\Delta/2$. Pour chaque pixel, dans le même ordre que l'encodeur, le code reçu est d'abord remplacé par son niveau de reconstruction, puis dénormalisé :

$$\hat{e} = y_i \cdot \sigma_e + \mu_e$$

Ensuite, la prédiction $p[j, k]$ est calculée à partir des pixels déjà décodés, avec le même prédicteur que l'encodeur et la même initialisation à 128 pour le premier pixel. Le pixel est reconstruit en ajoutant l'erreur à la prédiction :

$$\hat{x}[j, k] = p[j, k] + \hat{e}$$

On retrouve ainsi l'image synthétisée, pixel par pixel.

4.4 ALLOCATION DU BUDGET DE BITS

L'image d'entrée est de taille 256×256 pixels ce qui donne un total de 65 536 pixels. En DPCM, chaque pixel est prédit et comparé avec l'image source, uniquement l'erreur de prédiction quantifiée est transmise. Le quantificateur utilisé possède 32 niveaux, c'est donc 5 bits/pixel. Cela représente donc 327 680 bits de données par image ($5 \cdot 65\,536 = 327\,680$). Pour les métadonnées tant qu'à eux les informations nécessaires d'être transmises sont : delta, sigma et la moyenne. Ceux-ci sont transmis une fois pour l'image en chiffre réel (32 bits), ils représentent donc 96 bits par image ($3 \cdot 32 = 96$). La taille totale à transmettre est donc de $327\,680 + 96 = 327\,776$ bits.

$$R = \frac{\text{Taille totale en bits}}{\text{Nombre total de pixels}} = \frac{327\,776}{65\,536} = 5,0015 \text{ bits/pixel}$$

5. TECHNIQUE BTC

5.1 SCHÉMA-BLOC

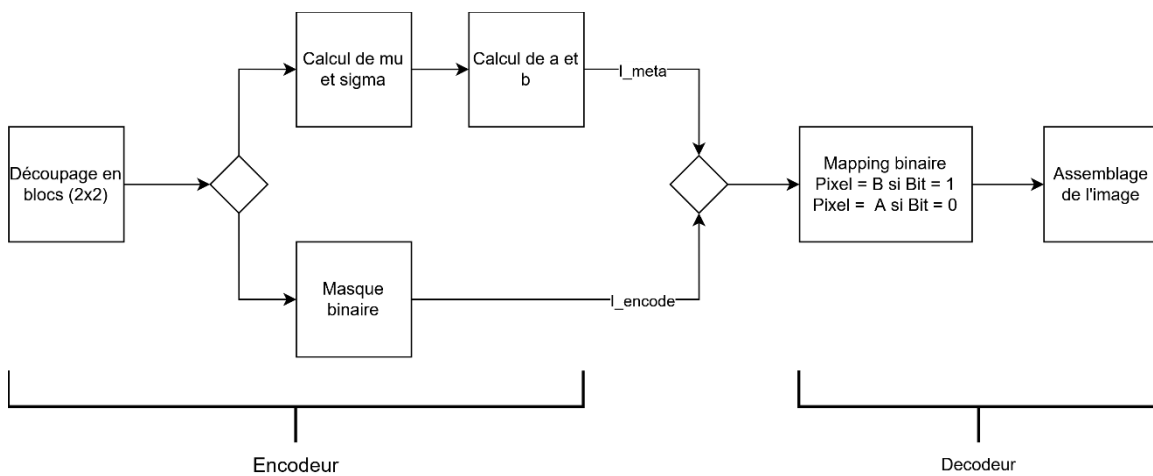


Figure 3 : Schéma bloc technique BTC

5.2 EXPLICATION DU CODEUR

Le codeur BTC commence par subdiviser l'image source en vecteurs de taille au choix. Dans ce cas, les subdivisions sont des blocs de 2×2 pixels. Ensuite, pour chacun des blocs, la moyenne

μ et l'écart-type σ sont calculés. La prochaine étape est de générer un masque binaire, c-à-d, comparer chaque pixel x_i du bloc à la moyenne μ du bloc de la manière suivante.

$$B_i = \begin{cases} 1 & \text{si } x_i \geq \mu \\ 0 & \text{si } x_i < \mu \end{cases}$$

Pour chaque bloc, la valeur de q est calculée, soit le nombre de pixels au-dessus de μ . Les niveaux de quantification a et b sont calculés de la manière suivante.

$$a = \mu - \sigma \sqrt{\frac{q}{m-q}} \quad b = \mu + \sigma \sqrt{\frac{q}{m-q}}$$

5.3 EXPLICATION DU DÉCODEUR

Pour le fonctionnement du décodeur, il faut envoyer les valeurs de quantification a et b de chaque bloc par métadonnée en plus du masque binaire de chaque bloc qui reconstitue l'image source. Avec ces deux éléments, chaque bloc l'image source peut être reconstruit en utilisant la formule suivante.

$$\hat{x}_i = \begin{cases} b & \text{si } B_i = 1 \\ a & \text{si } B_i = 0 \end{cases}$$

On retrouve ensuite l'image synthétisée de grandeur originale en juxtaposant chacun des blocs reconstruits.

5.4 ALLOCATION DU BUDGET DE BITS

L'image d'entrée est de taille 256×256 pixels ce qui donne un total de 65 536 pixels. Pour garder une bonne fidélité, les blocs de reconstitution sont petits, soit 2×2 pixels, ce qui donne un nombre total de 16 384 blocs. Sachant que chaque pixel est représenté par un bit, on a donc 4 bits par bloc et ainsi 65 536 bits par image. Il faut aussi transmettre les valeurs de quantification a et b pour chaque bloc. Pour représenter adéquatement une image en niveau de gris, il faut que les valeurs de quantification soit chacune 8 bits pour pouvoir représenter adéquatement les 256 tons qui constituent une image. Il faut donc 16 bits par bloc et ainsi 262 144 bits de métadonnée par image. La taille totale à transmettre est de donc 327 680 bits. Le débit binaire moyen est donc :

$$R = \frac{\text{Taille totale en bits}}{\text{Nombre total de pixels}} = \frac{327\,680}{65\,536} = 5 \text{ bits/pixel}$$

6. CONCLUSION

En conclusion, ce projet a permis de concevoir et de comparer trois approches de compression d'images adaptées aux exigences de la compagnie HyperMédia. Les techniques vectorielle, différentielle (DPCM) et par bloc (BTC) ont toutes démontré leur capacité à ramener le format de l'image source aux dimensions cibles tout en respectant le débit maximal prescrit de 5 bits/pixel, sauf la technique DPCM qui dépasse légèrement. Le choix final du codec dépendra du compromis souhaité entre la complexité et la qualité visuelle souhaitée ainsi que la mesure de PSNR. La méthode DPCM offre un bon compromis entre fidélité et complexité, tandis que la méthode vectorielle est complexe mais la plus fidèle. La méthode BTC offre la moins bonne fidélité, mais est la plus simple à implémenter.

Les figures Figure 4, Figure 5, Figure 6 montrent les images traitées des trois méthodes différentes et comparées à l'image source. Pour la même image, on voit un PSNR qui varie grandement selon la méthode de traitement. La méthode du DPCM est optimale, car elle offre un très bon PSNR et est moins compliqué que la méthode vectorielle. Comme on le voit au Tableau 1, le temps d'exécution est substantiellement plus rapide, en effet, l'opération se complète en 22 fois moins de temps. Bien qu'on dépasse légèrement la limite de 5 bits par pixel, on pourrait palier à ce problème en coupant l'envoi de quelques pixels pour revenir dans les limites et ce serait presque imperceptible à l'œil.

Tableau 1 : Tableau comparatif finale

	Temps exécution (sec)	Débit binaire (bit/pixel)	PSNR (dB)
DPCM	0,0845	5,0015	42,51
Vectoriel	1,8521	4,6250	47,93
BTC	0,1460	5,0000	35,62

7. IMAGES DE COMPARAISON LENNA



Figure 4 : Lenna version DPCM



Figure 5 : Lenna version vectorielle



Figure 6 : Lenna version BTC